tighten_cm_export_charset_baseline

tighten_cm_export_charset_baseline.sh — the CM-EXPORT-CHARSET-VALID producer

Revision: 2 **Last modified:** 2026-08-26T13:07:00Z **Authority:** §11.4.135 (monotone ratchet) · §11.4.249 (producer ≠ gate) · §11.4.240(B) (capability, not instruction) · §11.4.18 (script documentation)
Companion: `check_cm_export_charset_valid.md` — the gate this script feeds

Overview

This is the **only** component permitted to write the CM-EXPORT-CHARSET-VALID ratchet baseline, and it can only move it **down**.

It is a separate file for a structural reason, not a stylistic one. A gate that wrote its own threshold during a pre-build run would be a PRODUCER as well as a GATE — the collapse §11.4.240(C) names as the one that yields weakened thresholds. So the write capability lives here, in a script that is explicitly invoked by a human and never runs during a build. The gate contains no line that could call it and no line that could write the baseline.

The two share **one** oracle (`lib/cm_export_charset_scan.py`) and **one** baseline reader (`lib/cm_export_charset_baseline_read.sh`), so neither the number this script records nor the rule for what counts as a valid threshold can diverge from what the gate later enforces. The shared reader exists because the round-1 BLOCKING was that same parse rule written twice and fixed once.

Usage

```
# Lower the baseline to the live count. Refuses unless strictly
        lower.
bash scripts/pre_build/tighten_cm_export_charset_baseline.sh
        [SCAN_ROOT]

# Establish a baseline where none exists. First adoption only.
bash scripts/pre_build/tighten_cm_export_charset_baseline.sh --
        adopt [SCAN_ROOT]
```

`SCAN_ROOT` defaults to the project root. The baseline is always read from and written to `<SCAN_ROOT>/scripts/pre_build/cm_export_charset_valid.baseline` — the corpus and its threshold travel together, which is what lets test fixtures carry their own without any value-injecting environment variable.

When you will need it

The gate exits **3** with `STALE RATCHET` when the corpus has improved and the bar did not follow it down. That refusal names this command. Run it, commit the changed baseline, re-run the gate.

Exit codes

Exit	Meaning
`0`	baseline lowered, adopted, or already current (no-op)
`1`	refused — see the refusal table below
`2`	usage error

Every refusal, and why

Condition	Why it refuses
live count **above** stored baseline	That is a raise. The ratchet is monotone decrease (§11.4.135): a regression is fixed in the corpus, never accommodated by moving the bar.
stored baseline unparseable (`08`, `010`, `x`)	Refuses to overwrite a threshold it cannot first read. Leading zeros are the sharp case — bash

Condition	Why it refuses
	reads them as octal and *errors*, and `if` swallows the error as false, so before this was fixed the guards fell through and the script wrote `baseline=9` while printing "lowered 08 -> 9" (a raise, mislabelled).
stored baseline over 18 digits	Same class one ring out: the no-leading-zero rule accepts a 20-digit value and bash arithmetic wraps it mod 2^{64}, so `18446744073709551619` silently became 3. A number this script cannot faithfully evaluate is one it refuses to act on.
more than one `baseline=` line	An ambiguous threshold is an unresolvable signal, not something to resolve by picking the last matching line.
baseline path is a symlink	Refuses rather than silently replacing it with a regular file. A symlink can be re-pointed outside the tree, where later raises leave no diff — and quietly "healing" one would destroy whatever the operator meant by it.
baseline path is not a regular file	`mv` onto a directory succeeds by depositing the temp file *inside* it, leaving nothing at the path — while the script would report `ADOPTED` with rc 0. A success report for a write that did not happen is forbidden (§11.4.6).
post-write read-back disagrees	A zero exit from the writer proves bytes moved, never that they arrived at the intended path (§11.4.200). The value is read back and must match.
`--adopt` when a baseline exists	Blocks the careless raise. See "What the refusals do not do".

Condition	Why it refuses
no baseline and no `--adopt`	This mode lowers an existing ratchet; establishing one is a deliberate act.
blind scan (zero exports)	Tightening to a zero a blind scan produced would lock in a threshold nothing measured (§11.4.201(7)(b)).
no compliant export at all	The detector cannot be shown to discriminate.

What the refusals do not do (§11.4.6)

They raise the *cost* of a raise; they do not make one impossible. Delete the baseline and re-run `--adopt`, or edit the number by hand, and you have a higher bar. The `--adopt`-when-present refusal blocks the careless route, not the determined one. The **symlink** route is the exception — that one is refused outright, because it is the only route that goes trace-free after a single diff.

The actual defence is that **every one of those routes leaves a diff in a tracked file** for a reviewer to see (§11.4.142). Which means the defence exists only once the baseline file is **committed**. While it is untracked, no loosening route leaves a trace and the guarantee is void — which is why every successful write here prints “an untracked ratchet binds nothing (§11.4.215)”.

Output contract

The script reports what it **measured** and what it **wrote**, never a tighten it did not perform. On refusal it states the two numbers it compared. This is the anti-bluff contract its own header sets, and the F4 read-back exists because an earlier version broke it.

Related

- `scripts/pre_build/check_cm_export_charset_valid.sh` — the gate (read-only over the baseline)
- `scripts/pre_build/lib/cm_export_charset_scan.py` — the shared oracle

- `tests/pre_build/test_cm_export_charset_valid.sh` — §1.1 paired mutations covering both scripts
- `scripts/pre_build/lib/cm_export_charset_baseline_read.sh` — the shared read-only validator
- BOB-182 — the ratchet that did not ratchet